

Algorithms for solving the symmetry number problem on trees

Pradipta Prometheus Mitra^a, Muhammad Arshad Ul Abedin^b, Md. Abul Kashem^{b,*}

^a Yale University, New Haven, CT 06511, USA

^b Bangladesh University of Engineering and Technology, Dhaka 1000, Bangladesh

Received 30 July 2003; received in revised form 22 April 2004

Communicated by M. Yamashita

Abstract

The symmetry number of a tree is defined as the number of nodes of the maximum subtree of the tree that exhibits axial symmetry. Chin and Yen have presented an algorithm for solving the problem of finding the symmetry number of unrooted unordered trees in time $O(n^4)$. In this paper we present an improved algorithm for solving the symmetry number problem on trees that runs in time $O(n^3)$. We also show that our algorithm needs $O(n^2 \log n)$ time for trees with bounded degrees.

© 2004 Published by Elsevier B.V.

Keywords: Algorithms; Graph drawing; Symmetry number; Tree; Weighted matching

1. Introduction

Graph drawing is concerned with generating geometric representations of graphs. A main goal of graph drawing techniques is to construct aesthetic drawings of graphs. Various criteria on constructing pretty drawings of graphs have been discussed in literature [1]. Representative criteria include minimizing the number of edge crossings, minimizing the total drawing area, uniformity of edge length, minimizing the number of total bends, angular resolution, etc.

Symmetry is a rather new aesthetic criterion that has received attention recently [2,9]. Several types

of symmetry measures have been proposed. These include rotational and reflectional (axial) symmetries. Deciding whether a graph exhibits axial symmetry is, however, an intractable problem [2,9]. The *symmetry number* of an unrooted tree T is the number of nodes in the subtree of T that displays axial symmetry and has maximum cardinality [2]. The *symmetry number problem* is to find the symmetry number of a given tree T . Chin and Yen have presented an $O(n^4)$ time algorithm to compute the symmetry number of an unordered unrooted tree [2]. In this paper, we study the algorithm presented in [2] and propose some changes to that algorithm so that it runs in time $O(n^3)$. Furthermore, we show that the symmetry number of a tree with bounded maximum degree can be computed in time $O(n^2 \log n)$.

The organization of the paper is as follows. In Section 2, we discuss the definitions and existing

* Corresponding author.

E-mail addresses: pradipta.mitra@yale.edu (P.P. Mitra),
arshadul@dhaka.net (M. Arshad Ul Abedin),
kashem@cse.buet.ac.bd (Md.A. Kashem).

algorithms relevant to the discussion. In Sections 3 and 4, our approach to improve the algorithm is discussed. We present and prove a lemma that reduces redundant computations and then suggest use of better algorithms to solve the problem. In Section 5, we analyze the symmetry number problem on trees with bounded degrees and derive the relevant complexity results. In conclusion we summarize the study.

2. Existing algorithm for the symmetry number problem

The existing algorithm for computing the symmetry number of a tree is due to [2]. Some of the important definitions required to understand the problem and its solution as discussed in [2] are presented here.

Given a rooted tree $T = (V, E)$ and a node v , T_v is the rooted subtree of T that has v as root and contains all the vertices that are descendants of v . An r -subtree of the rooted tree T is a rooted tree generated from T by cutting off some of T 's subtrees. Tree T and any of its non-empty r -subtrees share the same root.

In [2], the authors solve the symmetry number problem by solving two subproblems. Two functions $A(v_i)$ and $B(v_i, v_j)$ are proposed to solve the problem. Function $A(v_i)$ returns the number of nodes in the maximum symmetric r -subtree of T_{v_i} subject to the constraint that v_i should be on the symmetry axis. Function $B(v_i, v_j)$ returns the number of nodes in the maximum r -subtree of T_{v_i} that is isomorphic to an r -subtree of T_{v_j} . The procedures for computing $A(v_i)$ and $B(v_i, v_j)$ are shown in Figs. 1 and 2 [2].

The correctness proof of these two procedures can be found in [2]. Computing $A(v_i)$ for all nodes $v_i \in V(T)$ can be done in $O(n^3)$ time, which is the

Procedure $A(v_i)$

```

1: if  $v_i$  is a leaf node then
2:   return 1;
3: else
4:   construct a weighted complete graph  $G_{v_i} = (V', E')$  with
     weight function  $w$  such that  $V' = \{v_i\} \cup C_{v_i}$ ,  $\{C_{v_i}\}$  is the set of
      $v_i$ 's children,  $w_{il} = A(v_l)$ ,  $\forall v_l \in C_{v_i}$ , and  $w_{pq} = 2B(v_p, v_q)$ ,
      $\forall v_p, v_q \in C_{v_i}$ ;
5:   return  $1 + (\text{weight of maximum matching of } G_{v_i})$ ;
6: end if

```

Fig. 1.

Procedure $B(v_i, v_j)$

```

1: if  $v_i$  or  $v_j$  is a leaf node then
2:   return 1;
3: else
4:   construct a weighted complete bipartite graph  $G_{v_i, v_j} =$ 
      $(V_i \cup V_j, E')$  with weight function  $w$  such that  $V_i = C_{v_i}$ ,
      $V_j = C_{v_j}$ , and  $w_{pq} = B(v_p, v_q)$ ,  $\forall v_p \in V_i$  and  $\forall v_q \in V_j$ ;
5:   return  $1 + (\text{weight of maximum matching of } G_{v_i, v_j})$ ;
6: end if

```

Fig. 2.

time needed for computing maximum matching on weighted graphs [8,10]. The complexity of a single call of $B(v_i, v_j)$ is $O(n_i n_j \sqrt{n} \log n)$, where $n_i = |C_{v_i}|$ and $n_j = |C_{v_j}|$, that is, the numbers of children of v_i and v_j , respectively. The complexity of computing $B(v_i, v_j)$ for all possible pairs for a single root is $O(n^{2.5} \log n)$ as shown in [2]. Thus for a rooted tree, computing the symmetry number needs $O(n^3 + n^{2.5} \log n) = O(n^3)$ time. Since each node $v \in V(T)$ can be considered as a root, computing the symmetry number of an unrooted tree T takes $O(n^4)$ time.

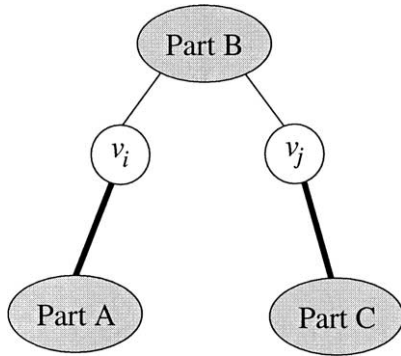
3. A theorem on computing $B(v_i, v_j)$

In [2], the authors mention that the function $B(v_i, v_j)$ must be computed once for each node of the tree T considering that node as the root of the tree. Thus they compute $B(v_i, v_j)$ in total n times. But, we claim that for a pair of nodes v_i and v_j , $v_i, v_j \in V(T)$, computing $B(v_i, v_j)$ only once is sufficient.

The notion that $B(v_i, v_j)$ should be computed once for each node as the root comes from the intuitive idea that the sets C_{v_i} and C_{v_j} would be different depending on the node that is considered as the root of the tree. We shall show that $B(v_i, v_j)$ is not computed for all other nodes taken as roots. It is only computed for certain roots and in all of those cases, the value is the same.

We divide the tree in three parts with respect to the two nodes v_i and v_j as shown in Fig. 3. Note that the bold lines connecting v_i with Part A and v_j with Part C may be a set of edges. However, lines connecting v_i and v_j with Part B denote strictly one edge.

The recursive calls in the related procedures start at either a certain root (when $A(v)$ is computed) or a pair of adjacent nodes (when $B(u, v)$ is directly

Fig. 3. Three parts of a tree T .

invoked). This pair of starting nodes will subsequently be referred to as the *bi-root*. We now present two lemmas required for the proof of Theorem 3.3.

Lemma 3.1. *The function $B(v_i, v_j)$ will never be required to be computed when the root or bi-root of the tree is completely in either Part A or Part C.*

Proof. Since Parts A and C are similar, it will be sufficient to prove for Part A. Whatever may be the assumed root or bi-root of the tree, v_i must be reached from that root or bi-root through the unique path (as nodes have a unique path between them in trees). As no path initiating from Part A can reach v_j without going through v_i , v_j will not be reached when v_i will be reached for the first time. Once v_i is reached, it is possible that v_j will be reached through subsequent recursive calls. But as v_i is a predecessor of v_j and because the recursion never pairs a predecessor with its successor, v_i will not be paired with v_j for computing $B(v_i, v_j)$. Nodes in Part A will never again reach v_i —the unique path from the root will not be retraced as children cannot invoke recursion involving their parents. $\square$

Lemma 3.2. *The value of $B(v_i, v_j)$ is unique if the root or bi-root is in Part B.*

Proof. There is a unique path between v_i and v_j . Part B is the collection of nodes from which there are paths to both v_i and v_j that do not include the other of the two. Obviously, all of such paths to v_i will go through the unique neighbor of v_i that lies on the unique path from v_i to v_j . Hence, the predecessor of v_i

is unique and so is the list of children. The same is true for v_j . As the children sets C_{v_i} and C_{v_j} are unique, so is $B(v_i, v_j)$. $\square$

Note that

- (1) the arguments in the proof of Lemma 3.2 hold trivially if any of the three parts is empty, because the proof has not assumed non-emptiness; and
- (2) the case where the recursive call starts from v_i itself is a special case of the case considered in Lemma 3.1. In this case v_j can be reached through recursive calls, but at that level v_i itself will not be reached as it is the root itself and edges from children to parents are never considered. A similar argument can be presented for v_j .

Theorem 3.3. *Given v_i and v_j , the values of all $B(v_i, v_j)$ required to be computed by the algorithm are the same.*

Proof. Immediate by Lemmas 3.1 and 3.2. $\square$

As $B(v_i, v_j)$ has to be calculated only once for each node pair, the total cost of computing all $B(v_i, v_j)$'s is $O(n^{2.5} \log n)$, instead of $O(n^{3.5} \log n)$.

4. Improved algorithms for computing $A(v_i)$

Though in the previous section, we have successfully decreased the cost of computing the function $B(v_i, v_j)$ from $O(n^{3.5} \log n)$ to $O(n^{2.5} \log n)$, that does not decrease the overall complexity of the algorithm. The bottleneck of computing all $A(v_i)$'s in time $O(n^4)$ means that the algorithm still runs in time $O(n^4)$. In this section, we first show that the use of more efficient algorithms for computing weighted matching improves the result of [2]. We then show how to improve the result further by using the matching update algorithm.

The proposed use of these algorithms are based on the observation that the graphs we are dealing with in this problem have integer weighted edges and the maximum total weight can be at most n , the number of nodes in the original tree. In [5], an $O(n^{3/4} m \log N)$ algorithm for solving the weighted matching problem for general graphs is presented, where N is

the maximum magnitude of an edge weight. Since $N \leq n$ and $m = O(n^2)$, clearly the total cost for computing all $A(v_i)$'s becomes $O(n^{3.75} \log n)$, which is asymptotically better than the previous $O(n^4)$ result of [2]. Furthermore, a better algorithm of time $O(\sqrt{(n\alpha(m, n) \log n)m \log(nN)})$ for solving the maximum matching problem is proposed in [6], where $\alpha(m, n)$ is the inverse of Ackermann's function [3]. Putting this result in [2] yields a better time complexity of $O(n^{3.5} \log^2 n)$.

To improve the performance of the algorithm more substantially, we propose to use the matching update algorithm [5–7].

The *matching update* problem [5] takes a complete structured matching and processes a sequence of update operations. The valid update operations are arbitrary changes to the graph at a node: edges incident to the node may be added, deleted or changed in weight. This problem is equivalent to finding a weighted augmenting path in the structured matching. In [4], it has been shown that a weighted augmenting path can be found in time $O(m + n \log n)$ which is $O(n^2)$ for a complete graph. We shall use this result to calculate all $A(v_i)$'s, $v_i \in V(T)$, in total time $O(n^3)$.

Lemma 4.1. *The total number of distinct $A(v_i)$'s, for all $v_i \in V(T)$ and all the possible roots, to be calculated is $3n - 2$.*

Proof. Let a node v_i have n_i neighbors. The value of $A(v_i)$ will depend on the root of the tree. When v_i itself is the root of the tree, the complete graph to calculate $A(v_i)$ will contain v_i and all its neighbors. If a neighbor of v_i is the root, the graph will consist of v_i and all its neighbors except for the neighbor that now is the root. So, considering only v_i or its neighbors as the parent, there are a total of $n_i + 1$ distinct $A(v_i)$'s.

Now, let us consider the value of $A(v_i)$'s for a root v_j which is neither v_i nor any of its neighbors. Surely, v_i must be reached from v_j through any one of v_i 's neighbors v_k . This value of $A(v_i)$ will be essentially equal to the value of $A(v_i)$ with v_k as the root. In other words, whatever the root of the tree, the parent of v_i must either be null (when it is itself the root), or one of its neighbors. So, the total number of distinct $A(v_i)$'s

is $n_i + 1$. Hence, total number of distinct $A(v_i)$'s (for the whole tree) is

$$\sum_{v_i \in V} (n_i + 1) = 2n - 2 + n = 3n - 2. \quad \square$$

Data structures:

- (1) For each $v \in V(T)$, a list L_v contains all possible values of $A(v)$. Thus $L_v[v_i]$ will denote the value of $A(v)$ with v_i as the parent of v in the rooted tree, and $L_v[v]$ is the value of $A(v)$ where v itself is the root of the tree. Initially all L_v 's are empty.
- (2) Let $f(u, v_i)$ be the parent of v_i when u is the root of the tree.
- (3) $G_{v_i} = (V'_{v_i}, E'_{v_i})$ is the complete weighted graph just as in the existing algorithm [2]. We say M_{v_i} as the data structures constructed while finding the maximum matching for G_{v_i} [3,5]. Both G_{v_i} and M_{v_i} are maintained during the execution of the algorithm and updated as the parent of v_i changes.

Outline of the algorithm for computing $A(v_i)$:

- (1) The value of $f(u, v)$ is calculated for each $u, v \in V(T)$ by one depth first search (DFS) for each $u \in V(T)$.
- (2) Starting from an arbitrary root u , $A(v_i)$ is calculated for each $v_i \in V(T)$ from scratch as described in [2], but the data structures for each node (used to calculate $A(v_i)$ for that node) are saved. The calculated $A(v_i)$ is stored in $L_{v_i}[f(u, v_i)]$.
- (3) For each node $v \in V(T)$ such that $v \neq u$, let v be the root of the tree. For each $v_i \in V(T)$, if $L_{v_i}[f(v, v_i)]$ has not been calculated yet, $A(v_i)$, with v as the root, is calculated by updating the data structure for maximum matching saved from the previous calculation. The $A(v_i)$ thus calculated is stored in $L_{v_i}[f(v, v_i)]$.

Before we present the algorithm in greater detail, we describe the procedures and their functionalities informally.

FindAllA: sets the global variable *root* to the current node and invokes $A(v)$ for all nodes $v \in V(T)$.

$A(v_i)$: if $A(v_i)$ has not been previously calculated, calculates it from scratch (as in [2], also

described in Section 2), otherwise updates the current data structures.

FindStructure(v_i): finds the weighted matching for the first time, and retains G_{v_i} and M_{v_i} constructed in the process.

UpdateCurrentA(v_i): uses the existing G_{v_i} and M_{v_i} to find the matching for the slightly changed graph.

Add(u, v_i) and *Delete*(v, v_i): adds and deletes the first node (u or v) from G_{v_i} and M_{v_i} .

The complete algorithm listings are shown in Figs. 4–9.

We present a small example to show how the algorithm works. Let us assume that there is a node u with neighbors v, w and x . By Lemma 4.1, we know that $A(u)$ will have only 4 distinct values, with each of u, v, w , and x as the root of the tree. Arbitrarily, let us assume that the first time $A(u)$ is invoked with v as the root. Since no structure currently exists, *FindStructure*(u) will be called, and it will calculate the maximum matching of the graph $G_u = (V'_u, E'_u)$ having the node set of $\{u, w, x\}$ (v being the parent will not appear in G_u , and G_u only includes u 's children in the rooted tree). Let us assume

Procedure FindAllA

```

1: for each  $u, v \in V(T)$  do
2:   calculate  $f(u, v)$ ;
3: end for
4: for each  $u \in V(T)$  do
5:    $root \leftarrow u$ ; { $root$  is a global variable}
6:   for each  $v \in V(T)$  do
7:     call  $A(v)$ ;
8:   end for
9: end for
```

Fig. 4.

Procedure A(v_i)

```

1: if  $L_{v_i}[f(root, v_i)] = nil$  then
2:   if no data structure exists for  $v_i$  then
3:      $L_{v_i}[f(root, v_i)] \leftarrow FindStructure(v_i)$ ;
4:   else
5:      $L_{v_i}[f(root, v_i)] \leftarrow UpdateCurrentA(v_i)$ ;
6:   end if
7: end if
8: return  $L_{v_i}[f(root, v_i)]$ ;
```

Fig. 5.

Procedure FindStructure(v_i)

```

1:  $C_{v_i} \leftarrow$  the set of  $v_i$ 's children;
2: if  $C_{v_i} = \emptyset$  then
3:    $G_{v_i} = (\{v_i\}, \emptyset)$ ; { $C_{v_i} = \emptyset$  implies that  $v_i$  is a leaf}
4:   return 1;
5: else
6:   construct a weighted complete graph  $G_{v_i} = (V'_{v_i}, E'_{v_i})$  with
     weight function  $w$  such that  $V'_{v_i} = \{v_i\} \cup C_{v_i}$ ,  $w_{il} = A(v_l)$ ,
      $\forall v_l \in C_{v_i}$ ,  $w_{pq} = 2B(v_p, v_q)$ ,  $\forall v_p, v_q \in C_{v_i}$ ;
7: find maximum matching of  $G_{v_i}$  and corresponding data
   structure  $M_{v_i}$ ;
8: return  $1 + (\text{weight of maximum matching of } G_{v_i})$ ;
9: end if
```

Fig. 6.

Procedure UpdateCurrentA(v_i)

```

1:  $V'' \leftarrow V'_{v_i}$ ; {previously constructed node set}
2:  $V' = \{v_i\} \cup C_{v_i}$ ; {this is the new node set}
3: if  $V'' \subset V'$  then
4:   let  $u \in V' - V''$ ; {it will be unique}
5:   Add( $u, v_i$ );
6: else
7:   if  $V' \subset V''$  then
8:     let  $v \in V'' - V'$ ; { $v$  will be unique}
9:     Delete( $v, v_i$ );
10:  else
11:    let  $u \in V' - V''$  and  $v \in V'' - V'$ ;
     {both  $u$  and  $v$  would be unique}
12:    Add( $u, v_i$ );
13:    Delete( $v, v_i$ );
14:  end if
15: end if
16: return  $1 + (\text{weight of maximum matching of } G_{v_i})$ ;
```

Fig. 7.

Procedure Add(u, v_i)

```

1: insert  $u$  into  $V'_{v_i}$ ;
2: insert  $(v_i, u)$  into  $E'_{v_i}$  with weight  $A(u)$ ;
3: for all other (except  $v_i$  and  $u$ )  $v_j \in V'_{v_i}$  insert  $(u, v_j)$  into  $E'_{v_i}$ 
   with weight  $B(v_j, u)$ ;
4: find the new maximum matching for the new  $G_{v_i}$  by updating
    $M_{v_i}$ ;
```

Fig. 8.

Procedure Delete(v, v_i)

```

1: delete  $v$  from  $V'_{v_i}$ ;
2: delete all  $(v, u) \in E'_{v_i}$  from  $E'_{v_i}$ ;
3: find the new maximum matching for the new  $G_{v_i}$  by updating
    $M_{v_i}$ ;
```

Fig. 9.

that the next time $A(u)$ is invoked with u itself as the root of the tree. Since there exists a structure, $UpdateCurrentA(u)$ will be called. The previous node set is $V'' = \{u, w, x\}$ and the new node set is $V' = \{u, v, w, x\}$ (it includes v as v is now a child of u , the root). Hence, $Add(v, u)$ will be called, which inserts edges (v, u) , (v, x) , and (v, w) with weights $A(v)$, $B(v, x)$, and $B(v, w)$, respectively. The matching for the resulting graph is found by the algorithm described in [4]. Next time if the root is w , then $Delete(w, u)$ will be called and V_u will become $\{u, v, x\}$, and the matching will be appropriately changed. Lastly, when x is the root, we must add w to the structure and delete x from it to find the new node set $\{u, v, w\}$ (and of course the corresponding edge and matching). This illustrates the “replace” situation in lines 11–13 of $UpdateCurrentA(v_i)$.

The uniqueness of u and v referred to in lines 4, 8 and 11 of $UpdateCurrentA(v_i)$ is easy to see. In $UpdateCurrentA(v_i)$, sets V' and V'' are node sets of v_i for two different roots. Any such set would include v_i and all of its neighbors except possibly the one which is currently the parent of v_i . If v_i itself is the root, then all neighbors are included. It is immediate that each of two such sets cannot have more than one node which is not contained in the other.

The correctness of the algorithm follows from the correctness of the previously known algorithms, since the function being computed is essentially the same.

Time complexity:

In lines 1–3 of procedure $FindAllA$, $f(u, v)$'s are calculated for all $u, v \in V(T)$.

As we have stated before, this can be done by n depth-first searches, resulting in an $O(n^2)$ time complexity. In lines 4–9 of the same procedure, the nested for loop will cause $O(n^2)$ invocations of $A(v)$. But by Lemma 4.1 we know that there are only $3n - 2$ unique values. So, the condition in line 1 of procedure $A(v_i)$ will be satisfied in only $3n - 2$ cases. For all other cases procedure $A(v_i)$ would immediately return the stored value. Thus in line 8 there will be $O(n^2)$ returns in total with $O(n^2)$ overall cost. We now have to find the time complexity of calculating those $3n - 2$ unique values of $A(v_i)$.

For each $v_i \in V(T)$, the condition in line 2 in procedure $A(v_i)$ holds true only once, because once the procedure $FindStructure(v_i)$ is called, a data structure for v_i is created and in subsequent cases line 5

is executed instead. Since $FindStructure(v_i)$ calculates a maximum matching on a weighted graph with $O(n_i)$ nodes, and the values of all weights $w_{pq} = 2B(v_p, v_q)$, $\forall v_p, v_q \in C_{v_i}$, are pre-computed as Theorem 3.3 suggests, the time complexity of $FindStructure(v_i)$ is $O(n_i^3)$, where n_i is the number of children of v_i . Thus the cost of calling $FindStructure(v_i)$ in total is $\sum_{v_i \in V(T)} O(n_i^3) = O(n^3)$.

By Lemma 4.1, we know that there are $n_i + 1$ distinct $A(v_i)$'s. Hence, for any $v_i \in V(T)$, the procedure $UpdateCurrentA(v_i)$ is called n_i times. In $UpdateCurrentA(v_i)$, the main operation is either $Add(u, v_i)$ or $Delete(u, v_i)$ or both. Costs of both of these operations are $O(n_i^2)$ for a graph with $O(n_i)$ nodes [4]. Hence the cost for all calls of $UpdateCurrentA(v_i)$ for a single v_i is $O(n_i^3)$, and the total cost for all $v_i \in V(T)$ is $\sum_{v_i \in V(T)} O(n_i^3) = O(n^3)$.

Thus the total cost for calculating all $A(v)$'s for all $v \in V(T)$ is $O(n^3)$ which is the total time complexity of the algorithm.

5. Analysis for trees with bounded degrees

In this section we observe that the time complexity of the algorithm improves if we assume that all nodes of the tree have degrees less than or equal to k , for some constant k . As a matter of fact, in this case, the complexity of computing $B(v_i, v_j)$ happens to dominate the overall time complexity of the algorithm instead of the complexity of computing $A(v_i)$.

Finding a maximum matching on an n -node, m -edge bipartite graph takes time $O(\sqrt{nm} \log(nN))$ [2,7], where N is the maximum magnitude of an edge weight. Since $N \leq n$, the time required to compute $B(v_i, v_j)$ is $O(n_i n_j (1 + \sqrt{n_i + n_j}) \log n)$, where n_i and n_j are the numbers of children of v_i and v_j , respectively (see [2] for details). Since in this case $n_i, n_j = O(1)$, the time required to compute $B(v_i, v_j)$ becomes $O(\log n)$. Thus for all $v_i, v_j \in V(T)$, the time required to compute $B(v_i, v_j)$ is $O(n^2 \log n)$.

For computing $A(v_i)$, the maximum matching on a k -node complete graph can be constructed in $O(k^3)$ time, where k is the maximum degree of the given tree. Then the time required for computing a single v_i is $O(k^3) = O(1)$. Thus for all $v_i \in V(T)$, the time required to compute all $A(v_i)$'s is $O(n)$. On the other hand, calculating $f(u, v)$ for all u, v requires $O(n^2)$ time as before.

Thus the overall complexity of the algorithm for trees with bounded degrees is $O(n^2 \log n)$.

6. Conclusion

In this paper, we present an improved algorithm for solving the symmetry number problem on trees T that runs in time $O(n^3)$, where n is the number of nodes in T . The previously known best algorithm is due to Chin and Yen [2] and their algorithm runs in time $O(n^4)$. In their algorithm the authors calculated the values of the functions $A(v)$ and $B(u, v)$, $u, v \in V(T)$, from scratch each time the root changes. But we show that $A(v)$ and $B(u, v)$ need to be calculated from scratch only once, and when the root changes $A(v)$ can be found by updating the previous data structure and $B(u, v)$ need not be recalculated at all. We also show that our algorithm runs in time $O(n^2 \log n)$ for trees with bounded degrees.

Acknowledgements

We would like to express our gratitude to the anonymous referees for their valuable comments and

suggestions which helped us to improve the presentation of the paper.

References

- [1] G. Battista, P. Eades, R. Tamassia, I. Tollis, *Graph Drawing: Algorithms for Visualization of Graphs*, Prentice Hall, New York, 1999.
- [2] K. Chin, H. Yen, The symmetry number problem for trees, *Inform. Process. Lett.* 79 (2001) 73–79.
- [3] T.H. Cormen, C.E. Leiserson, R.L. Rivest, *Introduction to Algorithms*, MIT Press, Cambridge, MA, 1998.
- [4] H. Gabow, Data structures for weighted matching and nearest common ancestors with linking, in: *Proc. of the 1st Annual ACM–SIAM Symp. on Discrete Algorithms*, 1990, pp. 434–443.
- [5] H.N. Gabow, A scaling algorithm for weighted matching on general graphs, in: *Proc. of the 26th Annual Symposium of the Foundations of Computer Science*, 1985, pp. 90–100.
- [6] H.N. Gabow, R.E. Tarjan, Faster scaling algorithms for general graph matching problems, *J. ACM* 38 (1991) 815–853.
- [7] H.N. Gabow, R.E. Tarjan, Faster scaling algorithms for network problems, *SIAM J. Comput.* 18 (5) (1989) 1013–1036.
- [8] E. Lawler, *Combinatorial Optimization: Networks and Matroids*, Holt, Rinehart and Winston, New York, 1976.
- [9] J. Manning, M. Atallah, Fast detection and display of symmetry in trees, *Congr. Numer.* 64 (1988) 159–169.
- [10] C.H. Papadimitrou, K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*, Prentice Hall, New York, 1982.